24DIT063
CSUE301 : Big Data Analytics

Practical 2

Aim:

Hadoop Installation, Configuration and HDFS Operations on Windows/Linux.

Problem Definition:

Scenario: You have recently joined DataAnalytics Inc. as a Hadoop Administrator. The organization processes large-scale customer, sales, and marketing data using the Hadoop ecosystem. Due to increasing data volumes, the company has decided to migrate its data storage infrastructure from traditional file systems to Hadoop Distributed File System (HDFS).

As a Hadoop Administrator, your responsibility is to install and configure Hadoop in a Single-Node Cluster environment on Windows/Linux, verify the Hadoop services, manage HDFS storage, and perform data management operations. Management wants a fully functional Hadoop environment that can store, retrieve, and process big data efficiently.

Objectives:

To install and configure Hadoop on a single-node cluster, access the Hadoop Web UI, and perform HDFS operations including directory creation, file upload/download, listing, viewing, copying, moving, renaming, and deleting files, along with cluster health monitoring and storage utilization checks for the Marketing Analytics team's big data environment.

Observation / Output (Screenshots or command outputs)

1. Verify Java installation (prerequisite for Hadoop).

`java -version`

```
aubaid-ahmed@Ubuntu-Coursework:~/hadoop/etc/hadoop$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu, mixed mode, sharing)
```

2. Verify Hadoop installation and check version.

`hadoop version`

```
aubaid-ahmed@Ubuntu-Coursework:~/hadoop/etc/hadoop$ cd ~
ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  f1.txt  hadoop  hadoop-3.4.2.tar.gz  hadoopdata  hdfs  index.html  snap
aubaid-ahmed@Ubuntu-Coursework:~$ nano ~/.bashrc
aubaid-ahmed@Ubuntu-Coursework:~$ nano ~/.bashrc
aubaid-ahmed@Ubuntu-Coursework:~$ source ~/.bashrc
aubaid-ahmed@Ubuntu-Coursework:~$ echo $JAVA_HOME
/usr/lib/jvm/java-11-openjdk-amd64
```

3. Configure Hadoop environment variables and core config files.

```
setx HADOOP_HOME "C:\hadoop"
setx PATH "%PATH%;%HADOOP_HOME%\bin;%HADOOP_HOME%\sbin"
# Edit core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml
```

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ echo $HADOOP_HOME
/home/aubaid-ahmed/hadoop
aubaid-ahmed@Ubuntu-Coursework:~$ hadoop version
Hadoop 3.4.2
Source code repository https://github.com/apache/hadoop.git -r 84e8b89ee2ebe6923691205b9e171badde7a495c
Compiled by ahmarsu on 2025-08-20T10:30Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum fa94c67d4b4be021b9e9515c9b0f7b6
This command was run using /home/aubaid-ahmed/hadoop/share/hadoop/common/hadoop-common-3.4.2.jar
```

```
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/core-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

4. Format the NameNode and start Hadoop services.

```
hdfs namenode -format
start-dfs.cmd
start-yarn.cmd
```

Output:

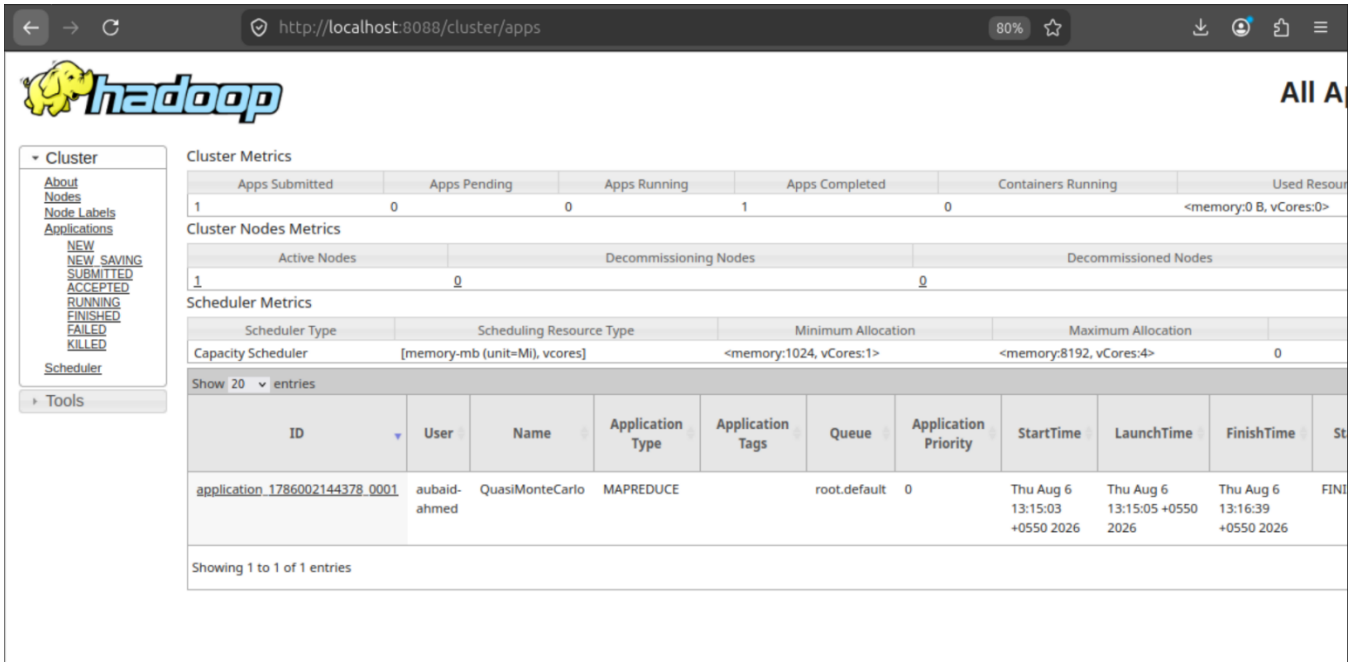
```
aubaid-ahmed@Ubuntu-Coursework:~$ rm /home/aubaid-ahmed/hadoop/etc/hadoop/yarn-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano /home/aubaid-ahmed/hadoop/etc/hadoop/yarn-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu-Coursework]
aubaid-ahmed@Ubuntu-Coursework:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
aubaid-ahmed@Ubuntu-Coursework:~$ jps
8064 SecondaryNameNode
7682 NameNode
8277 ResourceManager
7830 DataNode
8759 Jps
8408 NodeManager
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfsadmin -report
Configured Capacity: 32896749568 (30.64 GB)
Present Capacity: 15635963904 (14.56 GB)
DFS Remaining: 15635931136 (14.56 GB)
DFS Used: 32768 (32 KB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
```

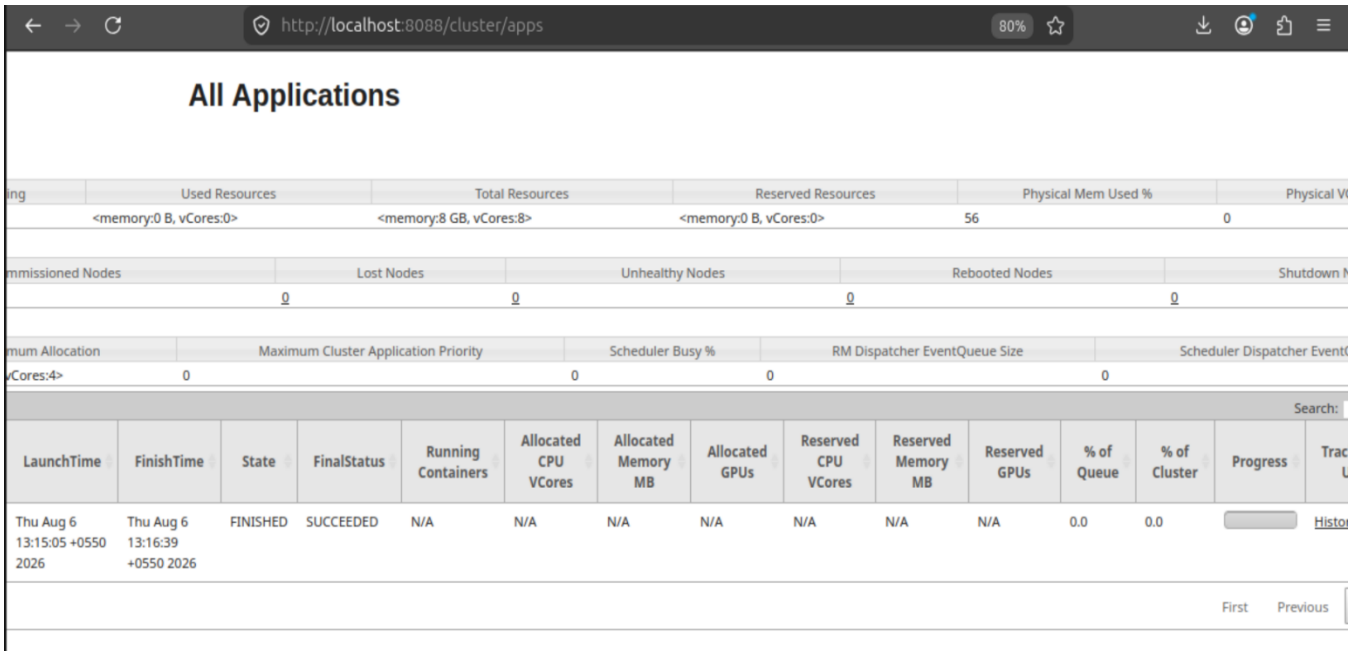
24DIT063
CSUE301 : Big Data Analytics

5. Access Hadoop Web UI.

```
# NameNode UI:   http://localhost:9870
# ResourceManager UI:  http://localhost:8088
```

Output:





6. Create directories in HDFS.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -mkdir -p /user/hadoop/input
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop
Found 1 items
drwxr-xr-x   - aubaid-ahmed supergroup          0 2026-08-06 21:04 /user/hadoop/input
```

7. Upload files to HDFS.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ echo "Hello Hadoop" > sample.txt
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -put sample.txt /user/hadoop/input
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/input
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:05 /user/hadoop/input/sample.txt
```

8. List HDFS files and directories.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/input
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:05 /user/hadoop/input/sample.txt
```

9. Display file contents.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -cat /user/hadoop/input/sample.txt
Hello Hadoop
```

10. Copy files within HDFS.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -mkdir /user/hadoop/backup
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -cp /user/hadoop/input/sample.txt /user/hadoop/backup/
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/backup
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:06 /user/hadoop/backup/sample.txt
```

11. Move files within HDFS.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/backup
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:06 /user/hadoop/backup/sample.txt
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -mkdir /user/hadoop/archive
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -mv /user/hadoop/input/sample.txt /user/hadoop/archive/
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/archive
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:05 /user/hadoop/archive/sample.txt
```

12. Rename files.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -mv /user/hadoop/archive/sample.txt /user/hadoop/archive/data.txt
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/archive
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:05 /user/hadoop/archive/data.txt
```

13. Download files from HDFS.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -get /user/hadoop/archive/data.txt .
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -ls /user/hadoop/archive
Found 1 items
-rw-r--r--   1 aubaid-ahmed supergroup         13 2026-08-06 21:05 /user/hadoop/archive/data.txt
```

14. Delete files and directories.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ ls
Desktop    Downloads  Pictures  Templates  data.txt  hadoop              hadoopdata  index.html  snap
Documents  Music      Public    Videos     f1.txt    hadoop-3.4.2.tar.gz hdfs        sample.txt
aubaid-ahmed@Ubuntu-Coursework:~$
aubaid-ahmed@Ubuntu-Coursework:~$
aubaid-ahmed@Ubuntu-Coursework:~$
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -rm /user/hadoop/archive/data.txt
Deleted /user/hadoop/archive/data.txt
```

15. Check HDFS report.

`hdfs dfsadmin -report`

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfsadmin -report
Configured Capacity: 32896749568 (30.64 GB)
Present Capacity: 15351070720 (14.30 GB)
DFS Remaining: 15350665216 (14.30 GB)
DFS Used: 405504 (396 KB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
```

```
-------------------------------------------------
Live datanodes (1):

Name: 127.0.0.1:9866 (localhost)
Hostname: Ubuntu-Coursework
Decommission Status : Normal
Configured Capacity: 32896749568 (30.64 GB)
DFS Used: 405504 (396 KB)
Non DFS Used: 15848456192 (14.76 GB)
DFS Remaining: 15350665216 (14.30 GB)
DFS Used%: 0.00%
DFS Remaining%: 46.66%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Thu Aug 06 21:09:05 IST 2026
Last Block Report: Thu Aug 06 21:03:59 IST 2026
Num of Blocks: 3
```

16. Verify block information.

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs fsck /user/hadoop -files -blocks -locations
Connecting to namenode via http://localhost:9870/fsck?ugi=aubaid-ahmed&files=1&blocks=1&locations=1&path=%2Fuser%2Fhadoop
FSCK started by aubaid-ahmed (auth:SIMPLE) from /127.0.0.1 for path /user/hadoop at Thu Aug 06 21:09:15 IST 2026

/user/hadoop <dir>
/user/hadoop/archive <dir>
/user/hadoop/input <dir>

Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    3
 Total symlinks:                0

Replicated Blocks:
 Total size:    0 B
 Total files:   0
 Total blocks (validated):      0
 Minimally replicated blocks:   0
 Over-replicated blocks:        0
 Under-replicated blocks:       0
 Mis-replicated blocks:         0
 Default replication factor:    1
 Average block replication:     0.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0
 Blocks queued for replication: 0
```

```
Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Thu Aug 06 21:09:15 IST 2026 in 11 milliseconds


The filesystem under path '/user/hadoop' is HEALTHY
```

17. Check Hadoop cluster health.

```
hdfs dfsadmin -report
# or via Web UI: http://localhost:9870
```

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs fsck /
Connecting to namenode via http://localhost:9870/fsck?ugi=aubaid-ahmed&path=%2F
FSCK started by aubaid-ahmed (auth:SIMPLE) from /127.0.0.1 for path / at Thu Aug 06 21:09:25 IST 2026


Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    14
 Total symlinks:                0

Replicated Blocks:
 Total size:    344217 B
 Total files:   3
 Total blocks (validated):      3 (avg. block size 114739 B)
 Minimally replicated blocks:   3 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    1
 Average block replication:     1.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0
```

```
Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Thu Aug 06 21:09:25 IST 2026 in 3 milliseconds


The filesystem under path '/' is HEALTHY
```

18. Monitor Hadoop processes.

`jps`

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ rm /home/aubaid-ahmed/hadoop/etc/hadoop/yarn-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ nano /home/aubaid-ahmed/hadoop/etc/hadoop/yarn-site.xml
aubaid-ahmed@Ubuntu-Coursework:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu-Coursework]
aubaid-ahmed@Ubuntu-Coursework:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
aubaid-ahmed@Ubuntu-Coursework:~$ jps
8064 SecondaryNameNode
7682 NameNode
8277 ResourceManager
7830 DataNode
8759 Jps
8408 NodeManager
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfsadmin -report
Configured Capacity: 32896749568 (30.64 GB)
Present Capacity: 15635963904 (14.56 GB)
DFS Remaining: 15635931136 (14.56 GB)
DFS Used: 32768 (32 KB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
```

19. Check storage utilization.

`hdfs dfs -du -h /MarketingAnalytics`

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~$ hdfs dfs -df -h
Filesystem                Size   Used  Available  Use%
hdfs://localhost:9000  30.6 G  396 K     14.3 G    0%
aubaid-ahmed@Ubuntu-Coursework:~$
```

Result:

Hadoop was successfully installed and configured in a single-node cluster environment, and the Hadoop services (NameNode, DataNode, ResourceManager, NodeManager) were verified as running. The MarketingAnalytics directory structure was created in HDFS, files were uploaded, listed, viewed, copied, moved, renamed, downloaded, and deleted as required, and the cluster health, block information, and storage utilization were checked, demonstrating effective HDFS administration for the Marketing Analytics team's big data environment.

Key Questions / Analysis:

Q1. Why is Hadoop preferred for Big Data storage?

Hadoop is preferred for Big Data storage because it can store and process massive volumes of structured, semi-structured, and unstructured data across a cluster of low-cost commodity hardware. It uses the Hadoop Distributed File System (HDFS) to split large files into blocks and distribute them across multiple nodes, providing horizontal scalability, fault tolerance through data replication, and high throughput for batch processing. Unlike traditional systems that are limited by the storage and processing capacity of a single machine, Hadoop scales out simply by adding more nodes, making it cost-effective and reliable for enterprises like DataAnalytics Inc. that handle ever-growing customer, sales, and marketing data.

Q2. What is the role of NameNode and DataNode in HDFS?

The NameNode is the master node of HDFS. It manages the file system namespace, maintains metadata such as file names, permissions, and the mapping of file blocks to DataNodes, and coordinates client access to files. It does not store the actual data itself. The DataNodes are the worker/slave nodes that store the actual data blocks on local disks, perform read/write operations as instructed by clients, and periodically send heartbeats and block reports to the NameNode to confirm they are alive and to report the blocks they hold. Together, the NameNode's metadata management and the DataNodes' block storage enable HDFS to reliably store and retrieve large files across a distributed cluster.

Q.3 What is the difference between HDFS and traditional file systems?

Traditional file systems (like NTFS or ext4) are designed for a single machine, store files as contiguous blocks on one disk, and have limited scalability and no built-in fault tolerance beyond RAID. HDFS, on the other hand, is a distributed file system designed to run across a cluster of machines. It splits files into large blocks (typically 128 MB or 256 MB), replicates each block across multiple DataNodes (default replication factor of 3) for fault tolerance, and is optimized for high-throughput, write-once/read-many batch workloads rather than low-latency random access. This makes HDFS far more scalable and resilient to hardware failure than a traditional single-machine file system, at the cost of higher latency for small, frequent read/write operations.

Q.4 What are the advantages of distributed storage?

Distributed storage, as implemented in HDFS, offers several advantages: it provides horizontal scalability, allowing storage capacity to grow by simply adding more nodes; it ensures fault tolerance and high availability through data replication across multiple nodes, so the failure of one node does not result in data loss; it enables

parallel data processing, since computation can be moved close to where the data resides (data locality), improving performance for large-scale analytics; and it is cost-effective, as it runs on commodity hardware rather than expensive specialized storage systems. These advantages make distributed storage well suited for enterprise big data platforms handling large and rapidly growing datasets.

Supplementary Problems:

1. Configure Hadoop pseudo-distributed mode.

```
# core-site.xml
<property><name>fs.defaultFS</name><value>hdfs://localhost:9000</value></property>

# hdfs-site.xml
<property><name>dfs.replication</name><value>1</value></property>

# mapred-site.xml
<property><name>mapreduce.framework.name</name><value>yarn</value></property>

# yarn-site.xml
<property><name>yarn.nodemanager.aux-
services</name><value>mapreduce_shuffle</value></property>
```

Answer)

- Open the Hadoop configuration files (core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml).
- Add the required properties such as the default HDFS address, replication factor, MapReduce framework, and YARN shuffle service.
- Save the configuration files and restart the Hadoop services.
- Verify that HDFS and YARN start successfully in pseudo-distributed mode.

2. Change HDFS replication factor and observe results.

```
hdfs dfs -setrep -w 2 /MarketingAnalytics/RawData/sales_data.txt
```

Answer)

- Execute the hdfs dfs -setrep -w 2 command on the required HDFS file.
- Hadoop updates the replication factor from the previous value to **2**.
- Wait until the replication process completes.
- Observe the confirmation message indicating that the new replication factor has been successfully applied.

3. Upload a large dataset into HDFS.

```
hdfs dfs -put big_sales_dataset.csv /MarketingAnalytics/RawData/
```

Answer)

- Ensure that the dataset is available on the local system.
- Use the hdfs dfs -put command to copy the dataset into the /MarketingAnalytics/RawData/ directory in HDFS.
- Wait for the upload process to complete.
- Verify that the file is present in HDFS using the appropriate listing command.

4. Analyze HDFS storage utilization.

```
hdfs dfs -du -h /
hdfs dfsadmin -report
```

Answer)

- Run the hdfs dfs -du -h / command to view the storage used by files and directories.
- Execute hdfs dfsadmin -report to display detailed HDFS storage information.
- Observe the total capacity, used space, remaining space, and DataNode status.
- Analyze the output to understand the overall storage utilization of the Hadoop cluster.

Tools / Technology used:

Windows 10 / Windows 11, OpenJDK 11 or above, Hadoop 3.x, Winutils.exe, Command Prompt / PowerShell.

Total Hours:

Problem Definition & Installation Planning: 0.5 Hour

Hadoop Installation & Configuration: 1 Hour

HDFS Operations & Analysis: 0.5 Hour

Conclusion:

Through this practical, Hadoop was installed and configured in a single-node cluster on Windows/Linux, and the Hadoop Web UI was used to verify that the NameNode, DataNode, ResourceManager, and NodeManager services were running correctly. Core HDFS operations such as directory creation, file upload/download, listing, viewing, copying, moving, renaming, and deleting files were performed, along with cluster health checks, block verification, and storage utilization analysis. This hands-on exercise provided a strong foundation in Hadoop administration and distributed storage management, essential for handling the growing data needs of the Marketing Analytics team at DataAnalytics Inc.